F Y E O

Ammo Markets Security Review

Ammo Markets

July 2026
Version 1.0

Presented by:
FYEO Inc.
PO Box 147044
Lakewood CO 80214
United States

Security Level
Public

Table of Contents

Executive Summary

Overview

Ammo Markets engaged FYEO Inc. to perform an Ammo Markets Security Review.

The assessment was conducted remotely by the FYEO Security Team. Testing took place on July 08 - July 13, 2026, and focused on the following objectives:

- To provide the customer with an assessment of their overall security posture and any risks that were discovered within the environment during the engagement.
- To provide a professional opinion on the maturity, adequacy, and efficiency of the security measures that are in place.
- To identify potential issues and include improvement recommendations based on the results of our tests.

This report summarizes the engagement, tests performed, and findings. It also contains detailed descriptions of the discovered vulnerabilities, steps the FYEO Security Team took to identify and validate each issue, as well as any applicable recommendations for remediation.

Key Findings

The following issues have been identified during the testing period. These should be prioritized for remediation to reduce the risk they pose:

- FYEO-AMO-01 – Cumulative oracle drift
- FYEO-AMO-02 – Daily mint capacity concentration
- FYEO-AMO-03 – Low-supply gvAMMO discounts
- FYEO-AMO-04 – Unlocked implementation initializer
- FYEO-AMO-05 – Unrecoverable direct AVAX
- FYEO-AMO-06 – Unregistered pool tax bypass
- FYEO-AMO-07 – Zero-value dust exits

Based on our review process, we conclude that the reviewed code implements the documented functionality.

Scope and Rules of Engagement

The FYEO Review Team performed a Ammo Markets Security Review. The following table documents the targets in scope for the engagement. No additional systems or resources were in scope for this assessment.

The source code was supplied through a public repository at https://github.com/ammo-markets/ammo-contracts with the commit hash 345d2fc8fae715f5f2ca66af6e22c1a19db3142c.

Remediations were submitted with the commit hash ce9c6d53a89a5d79217b29bf098da8e7f9676551.

Files included in the code review

```
ammo-contracts/
├── script/
│   ├── AddKeeper.s.sol
│   ├── DeployFuji.s.sol
│   ├── DeployFujiExchange.s.sol
│   ├── DeployFujiFullDev.s.sol
│   └── DeployMainnet.s.sol
└── src/
    ├── external/
    │   ├── exchange/
    │   │   ├── dev/
    │   │   │   └── DevAccessHub.sol
    │   │   ├── factories/
    │   │   │   └── PairFactory.sol
    │   │   ├── interfaces/
    │   │   │   ├── IAccessHub.sol
    │   │   │   ├── IERC20Extended.sol
    │   │   │   ├── IGauge.sol
    │   │   │   ├── IPair.sol
    │   │   │   ├── IPairCallee.sol
    │   │   │   ├── IPairFactory.sol
    │   │   │   ├── IRouter.sol
    │   │   │   ├── IVoter.sol
    │   │   │   └── IWETH.sol
    │   │   ├── libraries/
    │   │   │   ├── Errors.sol
    │   │   │   └── UQ112x112.sol
    │   │   ├── Pair.sol
    │   │   └── Router.sol
    │   └── gvToken/
    │       ├── interfaces/
```

Files included in the code review

```
│       │   └── IGvToken.sol
│       └── GvToken.sol
├── interfaces/
│   ├── IAmmoFactory.sol
│   ├── IAmmoManager.sol
│   ├── ICaliberMarket.sol
│   ├── ICaliberToken.sol
│   ├── IDexRouter.sol
│   ├── IPairFactory.sol
│   ├── ISolidlyPair.sol
│   └── IWETH.sol
├── AmmoFactory.sol
├── AmmoLiquidityManager.sol
├── AmmoManager.sol
├── CaliberMarket.sol
├── CaliberToken.sol
├── IPriceOracle.sol
├── MockUSDC.sol
└── PriceOracle.sol
```

Table 1: Scope

Technical Analyses and Findings

During the Ammo Markets Security Review, we discovered:

- 7 findings with INFORMATIONAL severity rating.

The following chart displays the findings by severity.

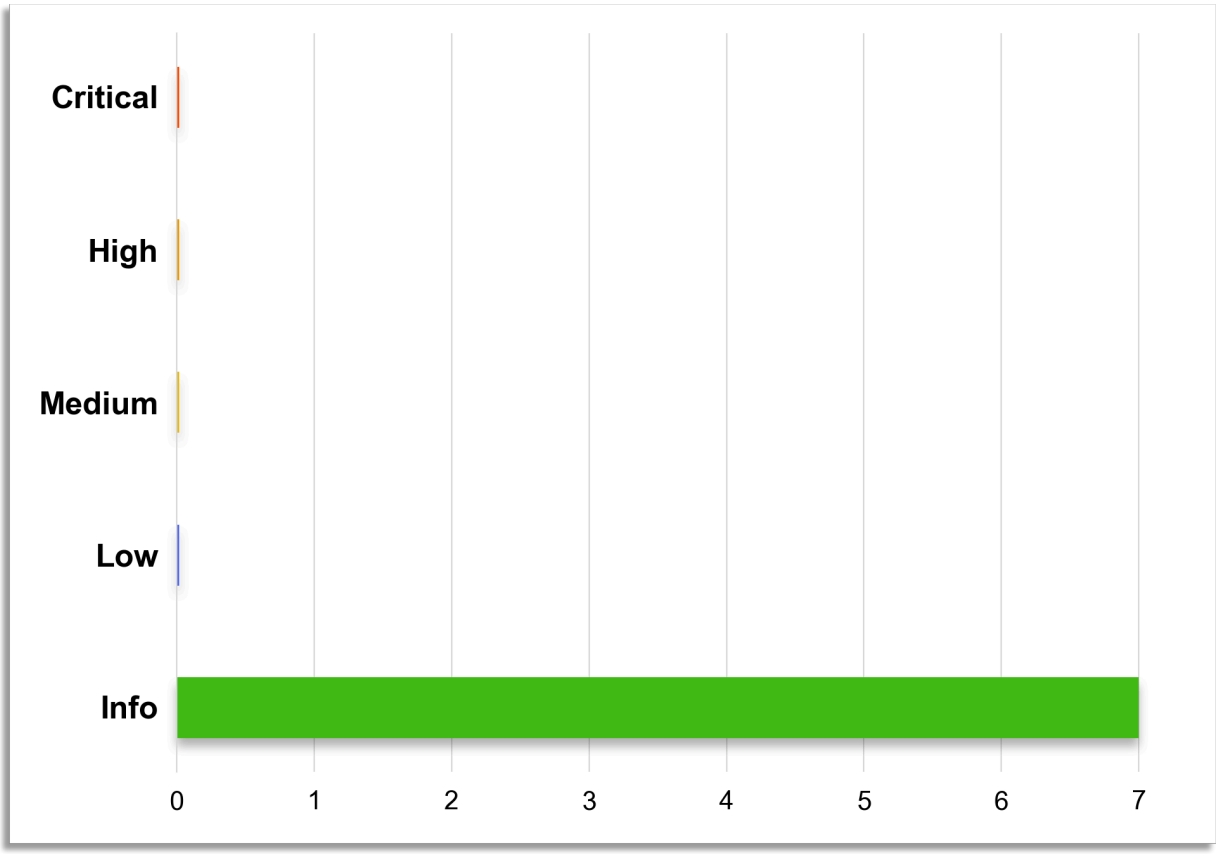

Figure 1: Findings by Severity

Findings

The *Findings* section provides detailed information on each of the findings, including methods of discovery, explanation of severity determination, recommendations, and applicable references.

The following table provides an overview of the findings.

Finding #	Severity	Description	Status
FYEO-AMO-01	Informational	Cumulative oracle drift	Remediated
FYEO-AMO-02	Informational	Daily mint capacity concentration	Acknowledged
FYEO-AMO-03	Informational	Low-supply gvAMMO discounts	Acknowledged
FYEO-AMO-04	Informational	Unlocked implementation initializer	Acknowledged
FYEO-AMO-05	Informational	Unrecoverable direct AVAX	Remediated
FYEO-AMO-06	Informational	Unregistered pool tax bypass	Acknowledged
FYEO-AMO-07	Informational	Zero-value dust exits	Remediated

Table 2: Findings Overview

The Classification of vulnerabilities

Security vulnerabilities and areas for improvement are weighted into one of several categories using, but is not limited to, the criteria listed below:

Critical – vulnerability will lead to a loss of protected assets

- This is a vulnerability that would lead to immediate loss of protected assets
- The complexity to exploit is low
- The probability of exploit is high

High - vulnerability has potential to lead to a loss of protected assets

- All discrepancies found where there is a security claim made in the documentation that cannot be found in the code
- All mismatches from the stated and actual functionality
- Unprotected key material
- Weak encryption of keys
- Badly generated key materials
- Txn signatures not verified
- Spending of funds through logic errors
- Calculation errors overflows and underflows

Medium - vulnerability hampers the uptime of the system or can lead to other problems

- Insecure calls to third party libraries
- Use of untested or nonstandard or non-peer-reviewed crypto functions
- Program crashes, leaves core dumps or writes sensitive data to log files

Low – vulnerability has a security impact but does not directly affect the protected assets

- Overly complex functions
- Unchecked return values from 3rd party libraries that could alter the execution flow

Informational

- General recommendations

Technical Analysis

The source code has been manually validated to the extent that the state of the repository allowed. The validation includes confirming that the code correctly implements the intended functionality.

Conclusion

Based on our review process, we conclude that the code implements the documented functionality to the extent of the reviewed code.

Technical Findings

General Observations

Ammo Markets is a protocol for representing real-world ammunition as blockchain tokens on Avalanche. Each supported ammunition caliber has its own market and token. Users deposit a stablecoin to request newly issued ammunition tokens, which authorized operators process after arranging the corresponding physical inventory. Token holders can later redeem their tokens for physical ammunition or request a stablecoin exit. Prices are supplied by authorized operators, and daily mint limits restrict how much new activity each market can accept.

The protocol includes centralized controls for treasury management, operator permissions, emergency pausing, transfer restrictions, and market configuration. Ammunition tokens can be traded through decentralized exchanges, where configured pools may charge transfer taxes that support the protocol treasury. A dedicated liquidity helper allows users to add or remove liquidity without unintended transfer taxes. Most user requests are asynchronous, allowing operators time to verify payment, inventory, and fulfillment before completing the transaction.

Cumulative oracle drift

Finding ID: FYEO-AMO-01
Severity: **Informational**
Status: **Remediated**

Description

`PriceOracle` limits each keeper update to 10% and one update per 30 minutes, but each update re-anchors the limit to the latest price. A compromised keeper can therefore compound valid updates while keeping the price fresh. After 12 windows, the price can reach approximately 3.14× upward or 0.28× downward.

This is unavailable to external attackers because every update requires the keeper role. The same trusted role controls settlement and funds exit payouts. Mint exposure also requires paid USDC and is bounded by the market's daily cap.

Proof of Issue

File name: src/PriceOracle.sol
Line number: 122

```
function _validatePriceChange(address market, uint256 oldPrice, uint256 newPrice)
private {
    uint256 change = oldPrice > newPrice ? oldPrice - newPrice : newPrice - oldPrice;
    if (change * DIVISOR > oldPrice * PRICE_CHANGE_THRESHOLD) {
        emit PriceChangeThresholdExceeded(market, oldPrice, newPrice);
        revert PriceChangeExceedsThreshold();
    }
}
```

Severity and Impact Summary

This is a defense-in-depth limitation under keeper compromise, not an unprivileged exploit. Update throttling, daily mint caps, and guardian pausing provide reaction time but do not impose a cumulative bound.

Recommendation

Use off-chain deviation monitoring that triggers the guardian pause, or add a carefully chosen rolling anchor. Consider separating oracle and settlement keeper roles.

Daily mint capacity concentration

Finding ID: FYEO-AMO-02
Severity: **Informational**
Status: **Acknowledged**

Description

`startMint` consumes daily capacity when a request is created, so one user can escrow enough USDC to consume the remaining capacity and make later mints in that market revert for the current chain-day.

This is not a free or reliable denial of service. The order requires real USDC that cannot come from a same-transaction flash loan because it remains escrowed. An ordinary requester cannot cancel before the greater-than-24-hour deadline, while a keeper can process the order and sweep the full deposit to treasury.

Proof of Issue

File name: src/CaliberMarket.sol
Line number: 96

```
_consumeDailyMintCapacity(usdcAmount);
usdc.safeTransferFrom(msg.sender, address(this), usdcAmount);
```

Severity and Impact Summary

A funded user can front-run another mint or consume configured capacity, but cannot immediately self-cancel and risks full keeper processing. The effect is limited to new mints in one market and ends on keeper cancellation or the next chain-day reset.

Recommendation

No security-critical change is required. If fair access is a product requirement, consider an order-size limit or a refundable queue with the cap enforced during keeper processing.

Low-supply gvAMMO discounts

Finding ID: FYEO-AMO-03
Severity: **Informational**
Status: **Acknowledged**

Description

`gvAmmoTaxDiscountBps` bases a trader's discount on `balanceOf / totalSupply` and permits a 100% discount. If gvAMMO is enabled while supply is thin, an early small staker can own enough supply to trade tax-free. `GvToken` balances also grow over time while `_totalSupply` changes only on deposits, withdrawals, or an owner update, so reported shares need not remain synchronized.

This affects only optional DEX tax revenue. The owner must enable gvAMMO and choose the thresholds, no user principal is at risk, and the external GvToken component is not currently deployed.

Proof of Issue

File name: src/AmmoManager.sol
Line number: 222

```
uint256 shareBps = (gvBalance * BPS_DIVISOR) / gvSupply;
if (shareBps <= gvAmmoTaxDiscountStartBps) return 0;
if (shareBps >= fullDiscountBps) return BPS_DIVISOR;
```

Severity and Impact Summary

This is a launch-parameter and economic-design concern for an optional feature. Thin supply and the absence of an absolute-stake floor can disproportionately reduce tax revenue.

Recommendation

Require a minimum absolute stake or minimum total supply before discounts activate. Consider retaining a tax floor and using a denominator synchronized with account balances.

Unlocked implementation initializer

Finding ID: FYEO-AMO-04
Severity: **Informational**
Status: **Acknowledged**

Description

`GvToken` is UUPS-upgradeable, but its implementation has no constructor calling `_disableInitializers()`. The first direct `initialize` caller becomes owner of that contract instance.

This does not take over a correctly deployed ERC1967 proxy because proxy storage is separate, initialization can be atomic, and UUPS upgrades require proxy context. The risk arises only from an incorrect two-step or unproxied deployment, and this external component is not currently deployed as part of the protocol.

Proof of Issue

File name: src/external/gvToken/GvToken.sol
Line number: 67

```
function initialize(address _stakingToken) external initializer {
    __ERC1967Upgrade_init();
    __Ownable_init();
    __UUPSUpgradeable_init();
    stakingToken = IERC20Upgradeable(_stakingToken);
}
```

Severity and Impact Summary

This is deployment hardening rather than an exploitable takeover of a correctly initialized proxy or the current protocol.

Recommendation

Lock the implementation initializer and initialize an ERC1967 proxy atomically at deployment. If the contract will remain un-proxied, use constructor-based initialization instead.

Unrecoverable direct AVAX

Finding ID: FYEO-AMO-05
Severity: **Informational**
Status: **Remediated**

Description

`AmmoLiquidityManager.receive()` accepts native AVAX, but the contract has no rescue function. `addLiquidityETH` isolates each caller's refund from the pre-existing balance, while `removeLiquidityETH` forwards the exact unwrapped amount and does not normally leave dust.

Only accidental direct transfers or forcibly sent AVAX become permanently stuck. Normal liquidity operations do not expose another user's funds.

Proof of Issue

File name: src/AmmoLiquidityManager.sol
Line number: 29

```
receive() external payable {}
```

Severity and Impact Summary

This is a recoverability limitation for donated or accidentally transferred AVAX, with no theft or attacker-imposed loss.

Recommendation

Add a suitably authorized native and ERC20 rescue mechanism, or document that assets sent directly to the helper are unrecoverable.

Unregistered pool tax bypass

Finding ID: FYEO-AMO-06
Severity: **Informational**
Status: **Acknowledged**

Description

`CaliberToken` applies buy and sell tax only when `from` or `to` is a pool configured in `AmmoManager.tokenPoolTax`. Transfers through an unregistered pool are treated like ordinary untaxed transfers.

Anyone can create an external pool, but the creator must provide counter-asset liquidity, bear LP risk, and attract willing traders. No principal can be stolen; the impact is forgone tax revenue if an unregistered venue gains activity.

Proof of Issue

File name: src/CaliberToken.sol
Line number: 127

```
function _determineTax(address from, address to, uint256 amount) internal view returns
(uint256) {
    (uint256 buyBps,) = manager.tokenPoolTax(address(this), from);
    if (buyBps > 0) return _taxAfterGvAmmoDiscount(to, amount, buyBps);

    (, uint256 sellBps) = manager.tokenPoolTax(address(this), to);
    if (sellBps > 0) return _taxAfterGvAmmoDiscount(from, amount, sellBps);

    return 0;
}
```

Severity and Impact Summary

This is a limited tax-revenue bypass, not a custody or solvency issue. If taxation is intentionally restricted to approved venues, the behavior is expected.

Recommendation

Treat transfer taxation as best-effort and monitor and register active supported pools. Factory membership can automate registration for known factories but cannot identify every external pool without restricting ordinary transfers.

Zero-value dust exits

Finding ID: FYEO-AMO-07
Severity: **Informational**
Status: **Remediated**

Description

`requestExit` rejects only a zero token amount. `_exitQuote` converts tokens to 6-decimal USDC and applies the 95% payout rate using integer division, so a sufficiently small request can produce `payoutUsdc == 0`. If finalized, the escrowed tokens are burned without a USDC payment.

The user chooses and approves the amount, and no third party can impose this loss. At a price of 1 USDC per token, the affected amount is below 0.000002 token. A keeper may decline to finalize it, and the user can cancel after the deadline.

Proof of Issue

File name: src/CaliberMarket.sol
Line number: 228

```
uint256 payoutUsdc = _exitQuote(tokenAmount, price);

token.transferFrom(msg.sender, address(this), tokenAmount);
```

Severity and Impact Summary

The behavior causes only self-selected rounding dust loss and does not affect other users or protocol funds.

Recommendation

Reject exit requests whose quoted payout is zero, and optionally enforce a practical minimum exit amount.

Our Process

Methodology

FYEO Inc. uses the following high-level methodology when approaching engagements. They are broken up into the following phases.

Figure 2: Methodology Flow

Kickoff

The project is kicked off as the sales process has concluded. We typically set up a kickoff meeting where project stakeholders are gathered to discuss the project as well as the responsibilities of participants. During this meeting we verify the scope of the engagement and discuss the project activities. It's an opportunity for both sides to ask questions and get to know each other. By the end of the kickoff there is an understanding of the following:

- Designated points of contact
- Communication methods and frequency
- Shared documentation
- Code and/or any other artifacts necessary for project success
- Follow-up meeting schedule, such as a technical walkthrough
- Understanding of timeline and duration

Ramp-up

Ramp-up consists of the activities necessary to gain proficiency on the project. This can include the steps needed for familiarity with the codebase or technological innovation utilized. This may include, but is not limited to:

- Reviewing previous work in the area including academic papers
- Reviewing programming language constructs for specific languages
- Researching common flaws and recent technological advancements

Review

The review phase is where most of the work on the engagement is completed. This is the phase where we analyze the project for flaws and issues that impact the security posture. Depending on the project this may include an analysis of the architecture, a review of the code, and a specification matching to match the architecture to the implemented code.

In this code audit, we performed the following tasks:

1. Security analysis and architecture review of the original protocol
2. Review of the code written for the project
3. Compliance of the code with the provided technical documentation

The review for this project was performed using manual methods and utilizing the experience of the reviewer. No dynamic testing was performed, only the use of custom-built scripts and tools were used to assist the reviewer during the testing. We discuss our methodology in more detail in the following sections.

Code Safety

We analyzed the provided code, checking for issues related to the following categories:

- General code safety and susceptibility to known issues
- Poor coding practices and unsafe behavior
- Leakage of secrets or other sensitive data through memory mismanagement
- Susceptibility to misuse and system errors
- Error management and logging

This list is general and not comprehensive, meant only to give an understanding of the issues we are looking for.

Technical Specification Matching

We analyzed the provided documentation and checked that the code matches the specification. We checked for things such as:

- Proper implementation of the documented protocol phases
- Proper error handling
- Adherence to the protocol logical description

Reporting

FYEO Inc. delivers a draft report that contains an executive summary, technical details, and observations about the project.

The executive summary contains an overview of the engagement including the number of findings as well as a statement about our general risk assessment of the project. We may conclude that the overall risk is low but depending on what was assessed we may conclude that more scrutiny of the project is needed.

We report security issues identified, as well as informational findings for improvement, categorized by the following labels:

- Critical
- High
- Medium
- Low
- Informational

The technical details are aimed more at developers, describing the issues, the severity ranking and recommendations for mitigation.

As we perform the audit, we may identify issues that aren't security related, but are general best practices and steps that can be taken to lower the attack surface of the project. We will call those out as we encounter them and as time permits.

As an optional step, we can agree on the creation of a public report that can be shared and distributed with a larger audience.

Verify

After the preliminary findings have been delivered, this could be in the form of the approved communication channel or delivery of the draft report, we will verify any fixes within a window of time specified in the project. After the fixes have been verified, we will change the status of the finding in the report from open to remediated.

The output of this phase will be a final report with any mitigated findings noted.

Additional Note

It is important to note that, although we did our best in our analysis, no code audit or assessment is a guarantee of the absence of flaws. Our effort was constrained by resource and time limits along with the scope of the agreement.

While assessing the severity of the findings, we considered the impact, ease of exploitability, and the probability of attack. This is a solid baseline for severity determination.